

UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

Facultad de Ingeniería

Ingeniería en Ciencias y Sistemas

Catedrático Ing. Otto Escobar Leiva

Aux: Diego Josue Guevara Abaj

MANUAL DE USUARIO

Edificio Inteligente IoT con Raspberry Pi ARM64

Proyecto de Arquitectura de Computadores y Ensambladores 1
Segundo Semestre 2026

Integrantes grupo 2

Angel Rafael Juárez Higueros

Jesus Rudy Espinoza Pivaral

Sebastián Alejandro Estrada Martínez

Roger Alberto Rivera Alvarez

202501037

201020621

202505125

201800551

Introducción

Porque programar en **ARM64** a bajo nivel (lenguaje ensamblador) es fundamental para los estudiantes de sistemas porque elimina las capas de abstracción y te permite entender cómo funciona realmente el hardware. Los lenguajes de alto nivel como Python o Java te ocultan la realidad física de la computadora; el bajo nivel te obliga a enfrentarte a ella. Este manual describe el uso del sistema de edificio inteligente IoT desarrollado sobre una Raspberry Pi con Linux de 64 bits. El sistema integra sensores, actuadores, comunicación MQTT, MongoDB Atlas, un dashboard web y un módulo de procesamiento en ensamblador AArch64. Su propósito es permitir al usuario monitorear el edificio, responder ante condiciones ambientales o de emergencia y controlar determinados dispositivos local y remotamente.

Objetivos

Generales

El sistema permite adquirir lecturas de temperatura, humedad, gas/humo, distancia y nivel de luz; automatizar ventilación, iluminación y acceso; mostrar el estado global del edificio; registrar lecturas, eventos y comandos; y visualizar los resultados estadísticos calculados por el módulo ARM64.

Generales

1. Diseñar e implementar una capa de comunicación orientada a eventos mediante un Broker MQTT, asegurando la transmisión asíncrona, ligera y en tiempo real de las variables del entorno (temperatura y humedad) en formato JSON desde el nodo periférico hacia el backend.
2. Desarrollar un script de orquestación de datos en Python (Puente IoT) que se encargue de centralizar la suscripción de los canales de mensajería, enriquecer las lecturas de los sensores con marcas de tiempo (timestamps UTC) y automatizar su persistencia en una base de datos no relacional en la nube mediante MongoDB Atlas.
3. Construir una interfaz gráfica de usuario interactiva (Dashboard) empleando el framework Streamlit, que consulte de manera eficiente la base de datos documental para representar analíticas visuales, tendencias históricas y tablas de auditoría interna accesibles para el usuario final.

3. Componentes principales

Componente	Función
Raspberry Pi version 3	Unidad principal de procesamiento y control.
DHT11/DHT22	Mide temperatura y humedad.
MQ-2/MQ-135	Detecta gas o humo.
HC-SR04	Detecta distancia para el acceso automatizado.
LDR	Mide el nivel de iluminación.
Servomotor SG90	Controla la apertura y cierre de la puerta.
Buzzer y LED rojo	Indican una condición de emergencia.
Ventilador DC y LED	Respuesta ante temperatura alta.
LEDs de iluminación	Iluminan las distintas zonas de la maqueta.
LCD 16x2/20x4	Panel físico de información.
4 botones físicos	Control local de puerta, iluminación, buzzer y alertas.
MQTT / EMQX	Comunicación entre componentes.
MongoDB Atlas	Persistencia de lecturas, eventos, comandos, estados y resultados ARM64.

4. Puesta en marcha

1. Verifique que la Raspberry Pi esté alimentada y conectada a la red.
2. Compruebe que los sensores, actuadores, LCD y botones estén correctamente conectados.
3. Inicie el programa principal en Python.
4. Verifique que la comunicación MQTT esté activa mediante el broker EMQX.
5. Compruebe que las lecturas comiencen a almacenarse en MongoDB Atlas.
6. Abra el dashboard web desde el dispositivo autorizado.
7. Confirme que el dashboard muestre el estado global y las últimas lecturas.

5. Estados globales del edificio

El sistema utiliza tres estados principales:

Estado	Comportamiento
NORMAL	Todos los sensores se encuentran dentro de los rangos seguros. LED verde.
ADVERTENCIA	Temperatura alta, humedad fuera de rango u otra condición no crítica. LED amarillo.
EMERGENCIA	El sensor de gas/humo supera el umbral definido. LED rojo y buzzer.

Importante: mientras persista una condición de emergencia por gas o humo, el sistema no debe regresar automáticamente a NORMAL.

6. Uso del monitoreo ambiental

El sensor DHT11/DHT22 proporciona temperatura y humedad periódicamente. Las lecturas se muestran en el dashboard y se almacenan con fecha y hora.

8. Revise la temperatura y humedad actuales en el panel principal.
9. Consulte las gráficas para observar el comportamiento histórico.
10. Si la temperatura supera el umbral configurado, el ventilador y su indicador se activan automáticamente.
11. El edificio pasa a ADVERTENCIA y el evento queda registrado.
12. Cuando la condición vuelve al rango normal, la respuesta automática se desactiva.

7. Uso del sistema de detección de gas o humo

El sensor MQ-2/MQ-135 se monitorea continuamente. Cuando su lectura supera el umbral definido, se activa la respuesta de emergencia.

13. El buzzer se activa y se enciende el LED rojo.
14. El estado global cambia a EMERGENCIA.
15. El dashboard presenta la alerta.
16. El evento se almacena en MongoDB Atlas.
17. La puerta se abre automáticamente para simular una evacuación.
18. Una vez desaparecida la condición peligrosa, el usuario puede restablecer la alerta.

8. Uso del acceso automatizado

El HC-SR04 mide la distancia frente a la entrada. Si se detecta una persona u objeto dentro del rango configurado, el servomotor abre la puerta y se enciende el LED indicador. La puerta puede controlarse de tres formas:

- Automáticamente mediante el sensor ultrasónico.
- Desde el dashboard web.

- Mediante el botón físico del panel.

Después del tiempo configurado, la puerta se cierra automáticamente. El dashboard muestra ABIERTA o CERRADA.

9. Uso de la iluminación inteligente

El LDR mide la iluminación ambiental. En modo AUTOMÁTICO, las luces se encienden cuando el nivel de luz está por debajo del umbral y se apagan cuando existe iluminación suficiente.

19. Seleccione AUTOMÁTICO o MANUAL desde el dashboard.

20. En modo MANUAL, utilice los controles del dashboard para encender o apagar las luces.

21. El botón físico permite alternar entre los modos.

22. El dashboard muestra el modo actual y el estado de cada zona.

10. Panel de control físico

La pantalla LCD presenta de forma rotativa:

- Temperatura y humedad
- Nivel de gas o humo
- Distancia detectada
- Nivel de luz
- Estado de la puerta
- Estado global del edificio

Los cuatro botones físicos deben permitir:

23. Abrir/cerrar puerta

24. Cambiar iluminación AUTOMÁTICO/MANUAL

25. Silenciar el buzzer

26. Restablecer la alerta cuando ya no exista la condición peligrosa

11. Uso del dashboard web

El dashboard debe utilizarse como centro de supervisión y control remoto.

27. Panel principal: revise el estado global y las últimas lecturas.

28. Actuadores: observe puerta, luces, ventilador y alarma.

29. Gráficas: consulte temperatura, humedad, gas/humo, distancia y nivel de luz.

30. Controles remotos: abra/cierre la puerta, controle las luces, cambie el modo de iluminación, active/desactive el ventilador, silencie la alarma y restablezca la emergencia cuando corresponda.

31. Historial: revise eventos, comandos y resultados ARM64.

12. Módulo ARM64

El módulo ARM64 procesa lecturas reales de temperatura. Python genera datos.txt, ejecuta el programa ARM64 y posteriormente lee resultado.txt. Los cálculos de máximo, mínimo, promedio entero y cantidad de datos son realizados por ARM64.

Flujo de operación:

Python → datos.txt → Programa ARM64 → resultado.txt → Python → MongoDB Atlas → Dashboard

El archivo de entrada utiliza un valor por línea y termina con el carácter \$. Ejemplo:

23

25

21

27

\$

La salida esperada tiene el formato:

MÁX=27

MIN=21

AVG=24

COUNT=4

13. Comunicación MQTT

El sistema utiliza MQTT como mecanismo obligatorio de comunicación. Los topics definidos en el proyecto incluyen:

- IDENTIFICADORUNICO/edificio/sensores/temperatura
- edificio/sensores/humedad
- edificio/sensores/gas
- edificio/sensores/distancia
- edificio/sensores/luz
- edificio/actuadores/puerta
- edificio/actuadores/luces
- edificio/actuadores/ventilador
- edificio/actuadores/alarma
- edificio/estado/global
- edificio/control/remoto
- edificio/arm64/resultados

14. Información almacenada en MongoDB Atlas

Colección	Contenido
sensor_readings	Lecturas de temperatura, humedad, gas, distancia y luz con timestamp.
events	Alertas, emergencias y cambios de estado.
commands	Comandos enviados desde dashboard o panel físico.
arm64_results	Máximo, mínimo, promedio, cantidad de datos y timestamp.
system_status	Estado global actual del edificio.

15. Solución de problemas

Problema	Acción recomendada
No aparecen lecturas	Verifique alimentación, cableado, programa Python y conexión MQTT.
Dashboard sin datos	Compruebe conexión al broker MQTT y que Python esté publicando los topics.
No se guardan datos	Revise conexión y credenciales de MongoDB Atlas y la colección correspondiente.
La puerta no se mueve	Verifique alimentación del servo, conexión GPIO y comandos recibidos por MQTT.
El buzzer no se apaga	Use el botón de silencio o el control remoto; el restablecimiento de emergencia solo procede cuando la condición peligrosa ya no

ARM64 no genera resultados

existe.

Compruebe datos.txt, el carácter \$, permisos de ejecución, compilación y resultado.txt.

16. Recomendaciones de operación

- Realice pruebas de cada sensor y actuador antes de iniciar una demostración completa.
- Mantenga ordenado y seguro el cableado de la maqueta.
- No desconecte la Raspberry Pi durante operaciones de escritura en la base de datos.
- Verifique que el sistema tenga conectividad de red antes de utilizar el dashboard.
- Ante una emergencia simulada, priorice la verificación de la condición del sensor antes de restablecer la alerta.
- Conserve evidencia de funcionamiento del flujo Python–ARM64 y del sistema IoT.

17. Flujo general de uso

32. Encender y verificar la Raspberry Pi.

33. Iniciar el sistema Python.

34. Verificar conexión MQTT.

35. Verificar almacenamiento en MongoDB Atlas.

36. Abrir el dashboard.

37. Comprobar lecturas y estado global.

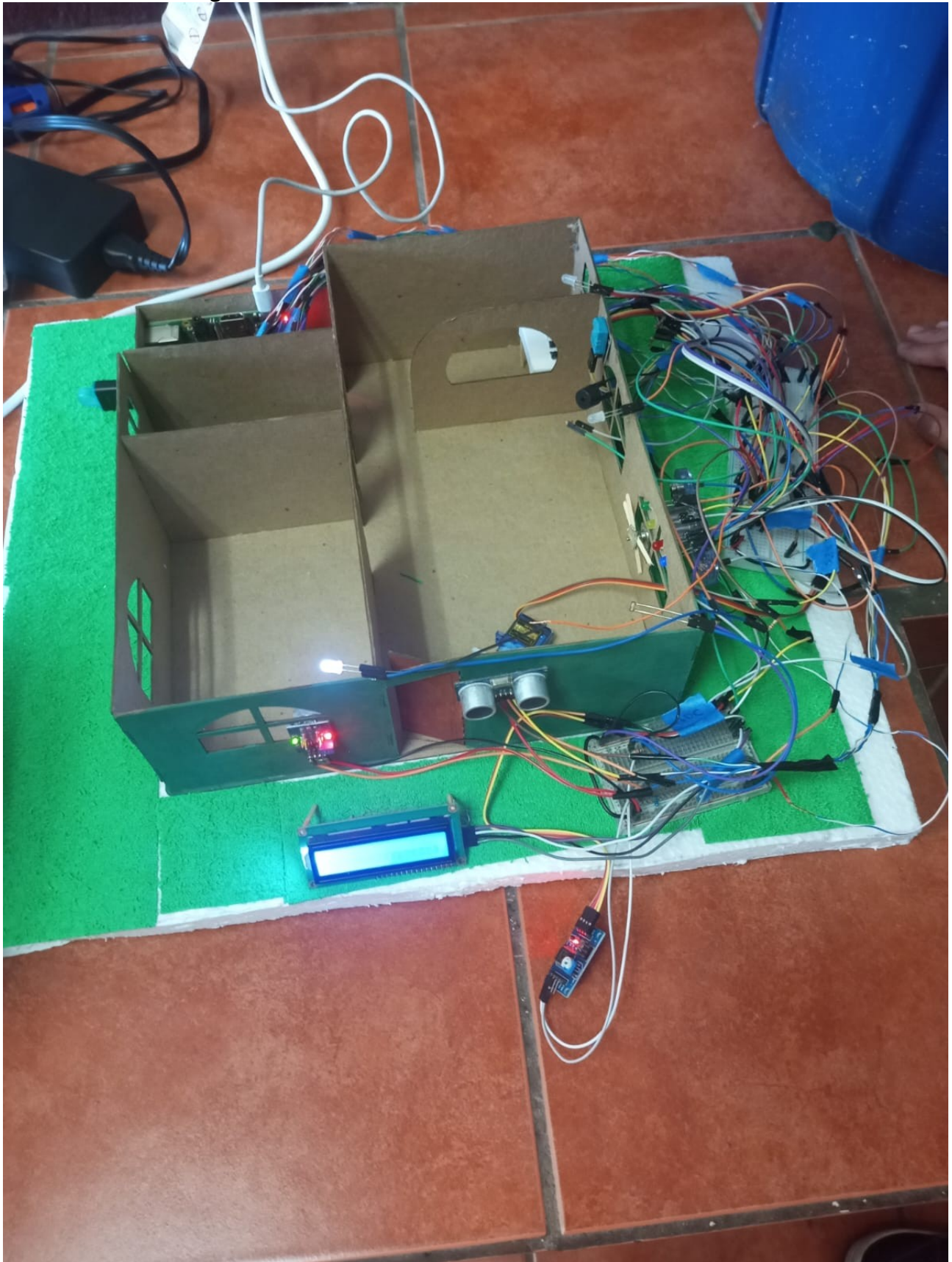
38. Probar controles de puerta, iluminación y ventilación.

39. Simular una condición de gas/humo y comprobar la respuesta de emergencia.

40. Verificar la generación de datos.txt y resultado.txt.

41. Comprobar que los resultados ARM64 aparezcan en MongoDB y dashboard.

Edificio/casa inteligente final



Referencias Bibliográficas

1. Documento oficial del proyecto “Edificio Inteligente IoT con Raspberry Pi ARM64”, correspondiente a Arquitectura de Computadores y Ensambladores 1, Segundo Semestre 2026.
2. AMR 64-Bit Assembly Language Larry D. Pyeatt with William Ughetta capitulo del 1 al 4
3. Materia de Apoyo de aux. Github/PoncheDeFrutas